Collection Operations & Data Pipelines Cheatsheet

Lessons: 54 — Filter, Map, Reduce, Group & Sets · 55 — Parsing & Shaping JSON, CSV & DB Rows

Examples: 54 · 55

Covers: the everyday collection shapes, generic helpers, `iter.Seq`, JSON/CSV/row parsing, ETL

Legend: `[*]` = API the lessons have not covered yet

GO HAS NO map/filter/reduce — write the loop

```
filter                      out := s[:0:0]                  (or make(..., 0, len(s)))
                            for _, v := range s { if keep(v) { out = append(out, v) } }
map                         out := make([]U, 0, len(s))
                            for _, v := range s { out = append(out, f(v)) }
reduce                      acc := zero
                            for _, v := range s { acc = f(acc, v) }
in-place filter             w := 0; for _, v := range s { if keep(v) { s[w]=v; w++ } }
                            s = s[:w]                        — no allocation
(the loop is idiomatic, obvious, and faster than a generic helper — reach for
 helpers only when the same shape repeats across the codebase)
```

THE GENERIC HELPERS (when you do want them)

```
func Filter[T any](s []T, keep func(T) bool) []T
func Map[T, U any](s []T, f func(T) U) []U
func Reduce[T, A any](s []T, init A, f func(A, T) A) A
func GroupBy[T any, K comparable](s []T, key func(T) K) map[K][]T
func CountBy[T any, K comparable](s []T, key func(T) K) map[K]int
func KeyBy[T any, K comparable](s []T, key func(T) K) map[K]T
func Partition[T any](s []T, pred func(T) bool) (yes, no []T)
func Chunk[T any](s []T, n int) [][]T
func Flatten[T any](ss [][]T) []T
func UniqueBy[T any, K comparable](s []T, key func(T) K) []T
(no stdlib home for these — a tiny internal/collections package is fine)
```

slices & maps (the stdlib toolbox)

```
slices.Contains / Index / IndexFunc / ContainsFunc
slices.Sort / SortFunc / SortStableFunc / IsSorted
slices.Max / Min / MaxFunc / MinFunc         panic on an EMPTY slice
slices.Compact(s)            drop ADJACENT duplicates — sort first for a real dedup
slices.CompactFunc(s, eq)
slices.Clone / Equal / EqualFunc
slices.Concat(a, b)          Go 1.22+
slices.Insert / Delete / Replace
slices.Reverse
maps.Keys(m) / maps.Values(m)    iter.Seq — wrap in slices.Sorted/Collect
maps.Clone / Equal / Copy / DeleteFunc
(slices.Sorted(maps.Keys(m)) is THE way to iterate a map in a stable order)
```

GROUPING, COUNTING & SETS

```
group by                    g := map[K][]T{}
                            for _, v := range s { k := key(v); g[k] = append(g[k], v) }
count by                    c := map[K]int{}; c[key(v)]++
key by (unique index)       idx := map[K]T{}; idx[key(v)] = v
set                         set := map[T]struct{}{}; set[v] = struct{}{}
  contains                  _, ok := set[v]
  union                     copy both into one map
  intersection              iterate the SMALLER one, test membership in the other
  difference                iterate a, keep what's absent from b
sum/avg per group           group first, then reduce each bucket
(a map is the answer to almost every "count/group/dedup/join" question)
```

ITERATORS (Go 1.23+)

```
iter.Seq[T]                 func(yield func(T) bool)
iter.Seq2[K, V]             func(yield func(K, V) bool)
for v := range seq          range-over-func
return false from yield     the consumer stops early; the producer must respect it
lazy filter                 func Filter[T any](s iter.Seq[T],
                              keep func(T) bool) iter.Seq[T] {
                              return func(yield func(T) bool) {
                                for v := range s {
                                  if keep(v) && !yield(v) { return }
                                }
                              }
                            }
slices.Values(s) / All(s)   slice -> iterator
slices.Collect(seq)         iterator -> slice
slices.Sorted(seq)          iterator -> sorted slice
maps.Keys(m) / maps.Values(m)     map -> iterator
why                         composable pipelines with no intermediate slices, and
                            a producer that can stream a file it never fully loads
```

PARSING JSON

```
json.Unmarshal(b, &v)       whole document in memory
json.NewDecoder(r).Decode(&v)     streaming from an io.Reader
`json:"name"` / `,omitempty` / `json:"-"`     the tags
*string / *int              nil distinguishes "absent/null" from the zero value
json.RawMessage             defer decoding a fragment
map[string]any              fully dynamic; numbers come back as float64
dec.DisallowUnknownFields()  catch typos and reject overposting
dec.More()                  a stream of concatenated values (NDJSON)
MarshalJSON / UnmarshalJSON  custom encoding for a type
json.Number              [*] keep numeric precision instead of float64
(only EXPORTED fields are encoded — an unexported one vanishes with no error)
```

PARSING CSV

```
r := csv.NewReader(f)
r.FieldsPerRecord = -1   [*] allow ragged rows
head, err := r.Read()        the header row
idx := map[string]int{}      name -> column index; NEVER hard-code positions
for { rec, err := r.Read(); if err == io.EOF { break } ... }
r.ReadAll()                  small files only
w := csv.NewWriter(f); w.Write(rec); defer w.Flush()     Flush or lose the tail
r.Comma = ';'            [*] a different delimiter
r.LazyQuotes = true      [*] survive malformed quoting
(index the header once — column order WILL change on you)
```

DB ROWS -> STRUCTS

```
rows, err := db.QueryContext(ctx, q, args...)
defer rows.Close()
for rows.Next() { var u User; rows.Scan(&u.ID, &u.Name); out = append(out, u) }
if err := rows.Err(); err != nil { ... }      the loop can end on an error
sql.NullString / sql.Null[T]      NULL-able columns
*string                       a pointer destination works too
rows.Columns()                dynamic result sets
(see the database sheet for the full API)
```

CONVERSION & TIME

```
strconv.Atoi / ParseInt / ParseFloat / ParseBool      always check the error
strconv.Itoa / FormatInt / FormatFloat
time.Parse("2006-01-02", s)  the reference layout, not placeholders
time.RFC3339                 the layout for APIs and logs
t.Format(layout)             back to text
empty string vs zero         decide what "" means BEFORE the parser does
(collect parse errors with the row number — "invalid input" alone is useless)
```

THE PIPELINE SHAPE

```
read      -> parse into records          (JSON / CSV / rows)
validate  -> collect errors with row numbers, don't stop at the first
filter    -> drop what you don't need, EARLY (less work downstream)
map       -> convert to a DTO/domain type
join      -> build map[K]T from the smaller side, then look up per row (hash join)
group     -> map[K][]T
aggregate -> reduce each bucket (sum, count, avg, min/max)
sort      -> slices.SortFunc with cmp.Or for multiple keys
paginate  -> slice the result, or keyset it at the source
write     -> encoder to an io.Writer, streaming
```

PIVOT, JOIN & STREAMING

```
hash join                    index the smaller dataset by key, scan the larger once
                             — O(n+m) instead of O(n*m)
left join                    a missing key means the zero value; decide and document
pivot / crosstab             map[rowKey]map[colKey]V, then a sorted key list per axis
NDJSON                       one JSON object per line — the streaming interchange format
  scanner per line, or json.Decoder + dec.More()
lazy record reader           func Records(r io.Reader) iter.Seq2[Record, error]
                             — process a 10GB file in constant memory
batching                     accumulate n records, flush, repeat
(stream by default: "read it all into a slice" is a memory limit you'll hit)
```

TRAPS & MEMORIZE

```
appending to a nil map        panic; make it first
map iteration order           random — sort the keys for any output a human reads
float64 for JSON numbers      big int64 IDs lose precision; use json.Number or a string
Compact without sorting       only removes ADJACENT duplicates
Max/Min on an empty slice     panic — check len first
csv.Writer without Flush      the last rows silently vanish
hard-coded CSV column indexes one inserted column breaks everything
ignoring rows.Err()           silently truncated results
unexported struct fields      never marshalled, never scanned, no error
reusing the loop var's address   &v across iterations (fine since 1.22, was a bug)
allocating in a hot pipeline  preallocate with make(..., 0, len(s))
loading the whole file        stream when you can't bound the input size
```